

Systemy czasu rzeczywistego, Projekt 3

Piotr Gugnowski, wydział EiTI, nr indeksu 320690

18 Maj 2026

Zadanie 2A — Pomiar czasów transmisji na wirtualnym porcie szeregowym

Cel zadania

Określenie możliwości zastosowania systemu operacyjnego do realizacji zadania komunikacyjnego w czasie rzeczywistym. Poszukiwanie rozwiązań praktycznych.

Uwaga: Zadanie realizowane na systemie macOS zamiast Windows.

Parametry transmisji

- Baudrate: 115200 bps
- Format ramki: 8E1 (8 bitów danych, bit parzystości, 1 bit stopu)
- Wysyłany bajt: 0xC4
- Wirtualny port szeregowy: para PTY tworzona przez `socat`

Parametry teoretyczne:

$$T_b = \frac{1}{115200} \approx 8,68 \mu s \quad (\text{czas bitu})$$

$$T_{znak} = 11 \cdot T_b \approx 95,49 \mu s \quad (1 \text{ start} + 8 \text{ data} + 1 \text{ parzystość} + 1 \text{ stop})$$

$$T_{gap_max} = 1,5 \cdot T_{znak} \approx 143,23 \mu s \quad (\text{limit MODBUS RTU})$$

We wszystkich podpunktach tryb blokowy opierał się na bloku wielkości 10. Liczba próbek w każdym pomiarze: $N = 10^6 - 1$.

a) Pomiar przy zamkniętych aplikacjach

Treść zadania

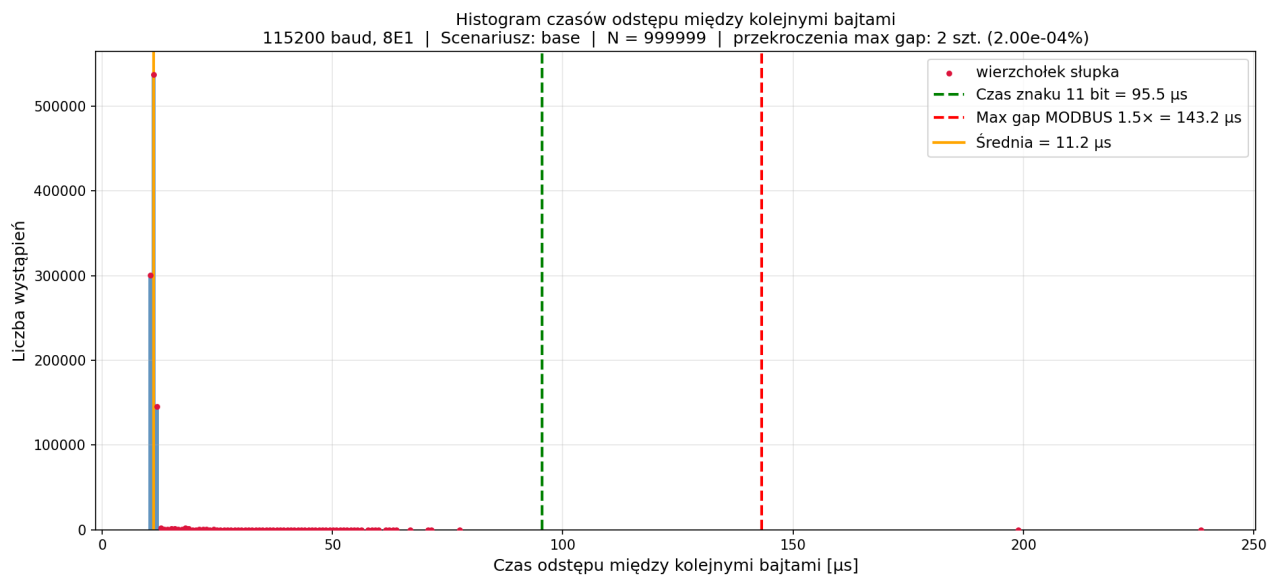
Napisać aplikację wysyłającą w sposób ciągły na wirtualny port szeregowy bajt 0xC4 z parametrami 115200, 8E1. Dokonać pomiaru odstępów czasowych między wysyłaniem kolejnych bajtów (metoda indywidualna oraz metoda blokowa). Sporządzić histogram częstości empirycznych czasów transmisji jednego znaku. Przed uruchomieniem zamknąć wszystkie aktywne aplikacje.

Rozwiązanie

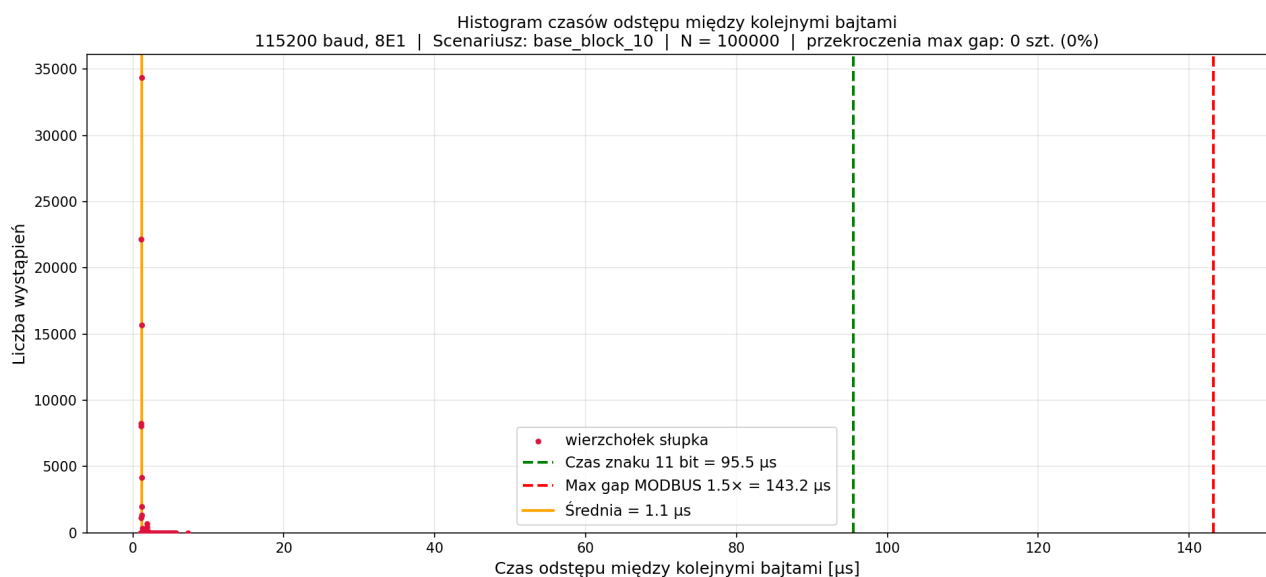
Aplikacja została zaimplementowana w języku Python. Kod źródłowy dostępny jest w pliku `zadanie2a.py`.

Zastosowano dwie metody pomiaru:

- **Metoda indywidualna** — każdy bajt wysyłany osobno przez `write(1B) + flush()`; mierzony jest czas między kolejnymi wywołaniami.
- **Metoda blokowa** — blok $n = 10$ bajtów wysyłany jednorazowo; mierzony jest łączny czas bloku, a następnie dzielony przez n w celu wyznaczenia średniego czasu na bajt.



Rysunek 1: Histogram czasów transmisji — metoda indywidualna, aplikacje zamknięte. Średni czas: $\bar{t} = 11,2 \mu s$, przekroczenia limitu MODBUS: 2.



Rysunek 2: Histogram czasów transmisji — metoda blokowa (blok = 10 B), aplikacje zamknięte. Średni czas: $\bar{t} = 1,1 \mu s$, przekroczenia limitu MODBUS: 0.

Średni czas w metodzie indywidualnej ($11,2 \mu s$) jest niemal dziesięciokrotnie wyższy niż w metodzie blokowej ($1,1 \mu s$). Różnica wynika z narzutu wywołań systemowych: w metodzie indywidualnej każdy bajt generuje osobną parę wywołań `write/tcdrain`, podczas gdy w metodzie blokowej ten narzut jest amortyzowany przez n bajtów. Na wirtualnym porcie (PTY) nie występuje rzeczywiste opóźnienie sprzętowe UART, więc dominuje narzut programowy OS. Oba wyniki są znacznie poniżej teoretycznego czasu znaku ($95,49 \mu s$), co potwierdza brak fizycznej transmisji.

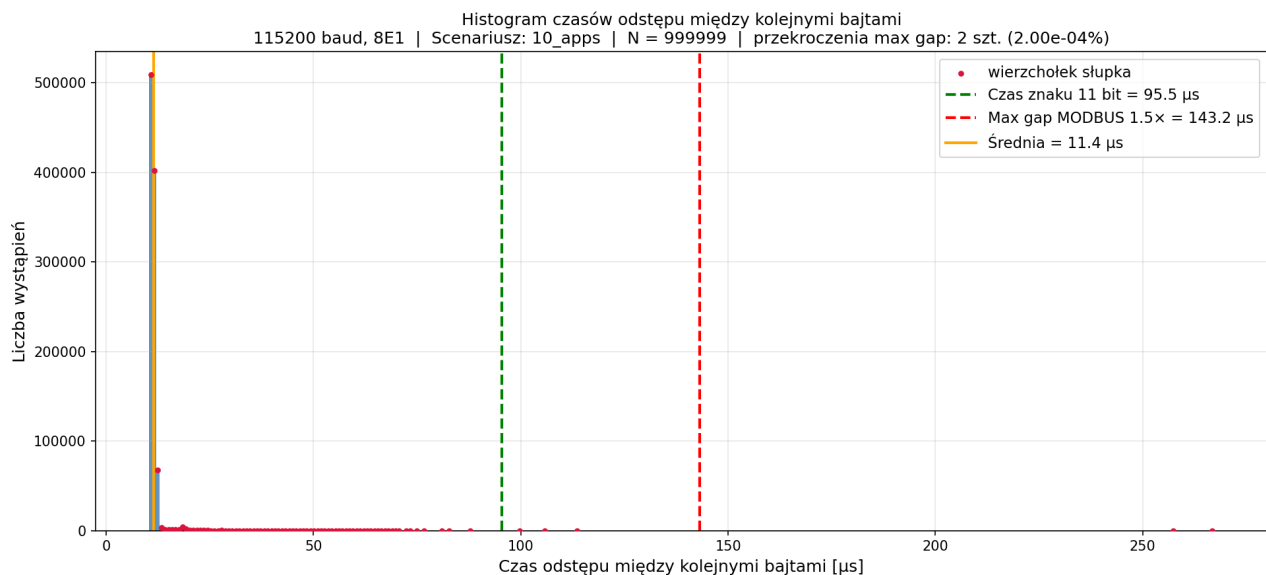
b) Pomiar przy 10 aktywnych aplikacjach

Treść zadania

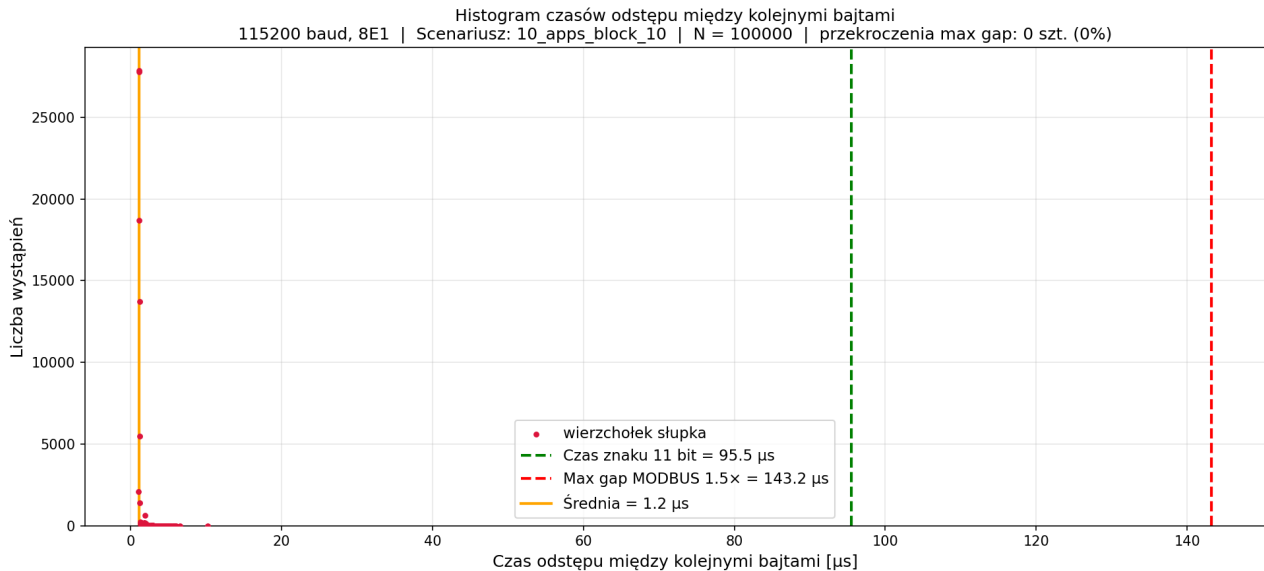
Wyznaczyć rozkład czasu transmisji przy aktywnych 10 aplikacjach. Czy średni czas transmisji 1 znaku ulegnie zmianie?

Rozwiązanie

Uruchomiono 10 aplikacji na komputerze i powtórzono oba pomiary.



Rysunek 3: Histogram czasów transmisji — metoda indywidualna, 10 aktywnych aplikacji. Średni czas: $\bar{t} = 11,4 \mu s$, przekroczenia limitu MODBUS: 2.



Rysunek 4: Histogram czasów transmisji — metoda blokowa (blok = 10 B), 10 aktywnych aplikacji. Średni czas: $\bar{t} = 1,2 \mu\text{s}$, przekroczenia limitu MODBUS: 0.

Średni czas transmisji uległ zmianie w obu przypadkach:

- metoda indywidualna: $11,2 \rightarrow 11,4 \mu\text{s}$ (wzrost o $\approx 1,8\%$),
- metoda blokowa: $1,1 \rightarrow 1,2 \mu\text{s}$ (wzrost o $\approx 9\%$).

Aktywne aplikacje zwiększają obciążenie schedulera, co przekłada się na większy jitter i nieznacznie wyższy średni czas między wysyłanymi bajtami. Liczba przekroczeń limitu MODBUS nie zmieniła się (2 w metodzie indywidualnej), co sugeruje, że wzrost obciążenia na poziomie 10 aplikacji jest jeszcze niewystarczający, by istotnie pogorszyć terminowość transmisji.

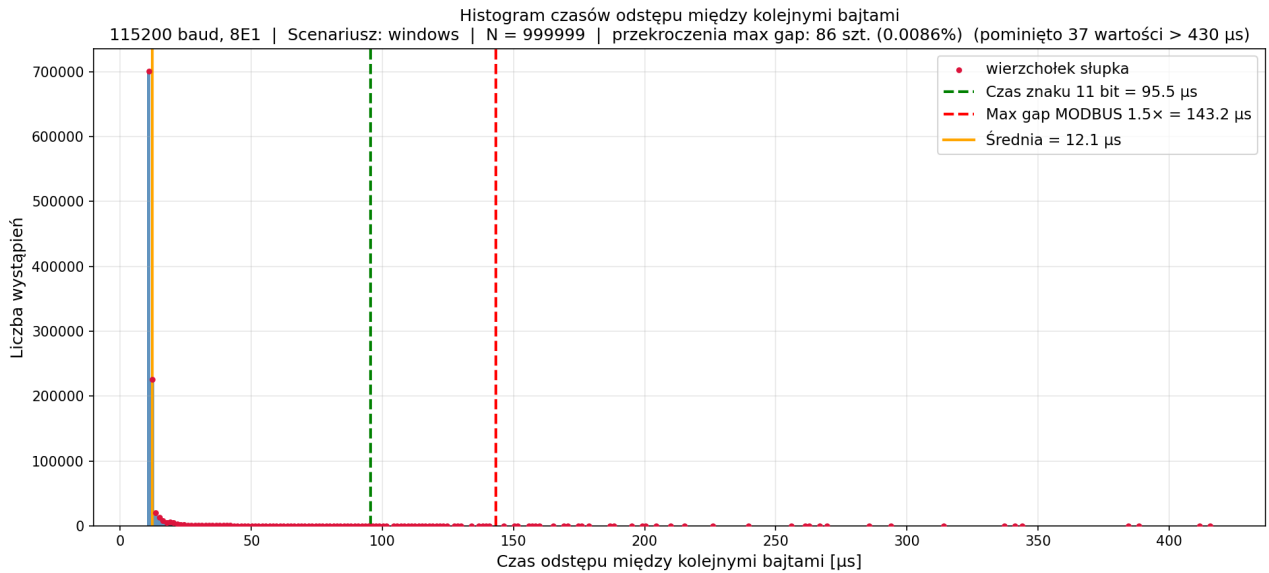
c) Wpływ operacji na oknach

Treść zadania

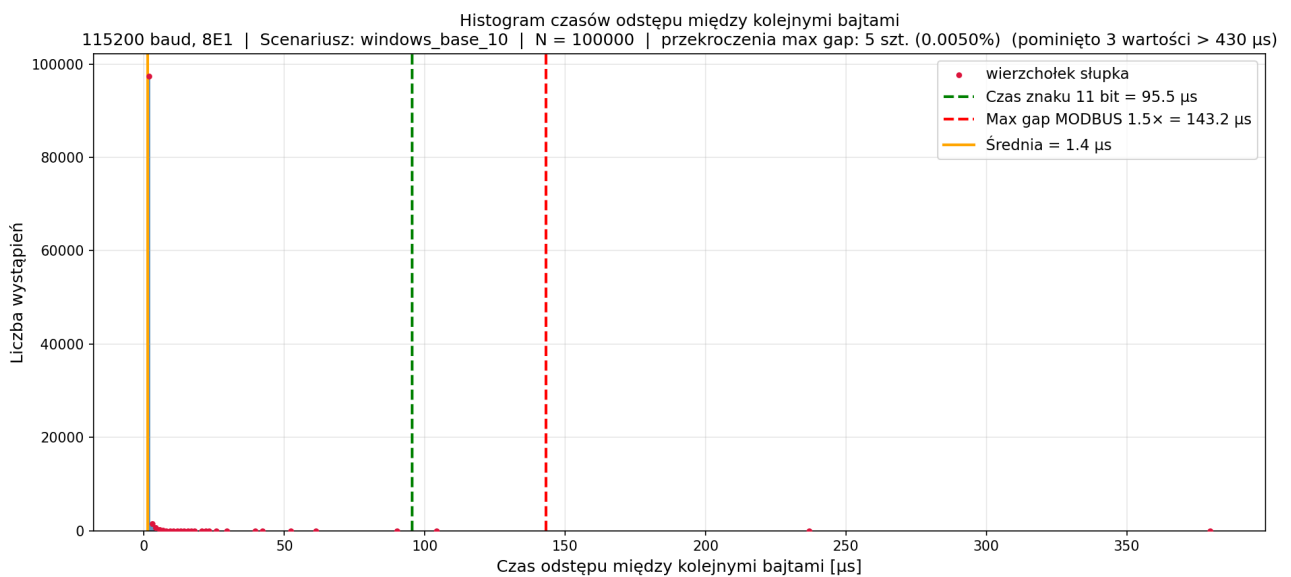
Zbadać wpływ operacji na oknach (ręczne przeciąganie i skalowanie okien) na średni czas transmisji 1 znaku.

Rozwiązanie

Podczas pomiaru wykonywano intensywne operacje na oknach systemu (przeciąganie, skalowanie).



Rysunek 5: Histogram czasów transmisji — metoda indywidualna, operacje na oknach. Średni czas: $\bar{t} = 12,1 \mu$ s, przekroczenia limitu MODBUS: 86.



Rysunek 6: Histogram czasów transmisji — metoda blokowa (blok = 10 B), operacje na oknach. Średni czas: $\bar{t} = 1,4 \mu$ s, przekroczenia limitu MODBUS: 5.

Operacje na oknach spowodowały wyraźne pogorszenie parametrów transmisji:

- metoda indywidualna: średni czas wzrósł o $\approx 8\%$ ($11,2 \rightarrow 12,1 \mu$ s), liczba przekroczeń wzrosła z 2 do **86**,
- metoda blokowa: średni czas wzrósł o $\approx 27\%$ ($1,1 \rightarrow 1,4 \mu$ s), pojawiło się **5** przekroczeń (wcześniej: 0).

Poruszanie oknami angażuje podsystem graficzny (GPU, compositor), co powoduje długie wyłączenia procesu pomiarowego przez system operacyjny. Skutkuje to gwałtownym wzrostem liczby przekroczeń limitu MODBUS RTU, szczególnie widocznym w metodzie blokowej — tryb blokowy nie zapewnia tu żadnej ochrony przed nieregularnościami schedulera.

System macOS, podobnie jak Windows, jest systemem operacyjnym ogólnego przeznaczenia (GPOS) i nie gwarantuje deterministycznych czasów odpowiedzi. Operacje graficzne znacząco obniżają terminowość transmisji szeregowej. W zastosowaniach hard real-time wymagany byłby dedykowany system czasu rzeczywistego (RTOS) lub sprzętowe wsparcie dla priorytetyzacji zadań czasowych.

Zadanie 2B — Pomiar odstępów między znakami w ramce MODBUS RTU

Cel zadania

Zbadanie rozkładu czasów odstępów między kolejnymi znakami w ramce protokołu MODBUS RTU, przy różnych obciążeniach systemu operacyjnego.

Uwaga: Zadanie realizowane na systemie macOS zamiast Windows.

Parametry transmisji

- Baudrate: 115200 bps, format 8E1
- Wysyłana ramka: 01 03 00 00 00 02 C4 0B (8 znaków)
- Metoda: wyłącznie zastępcza — $\bar{T}_{znak} = T_{ramka}/8$
- Liczba ramek w każdym pomiarze: $N = 10^6 - 1$

Parametry teoretyczne:

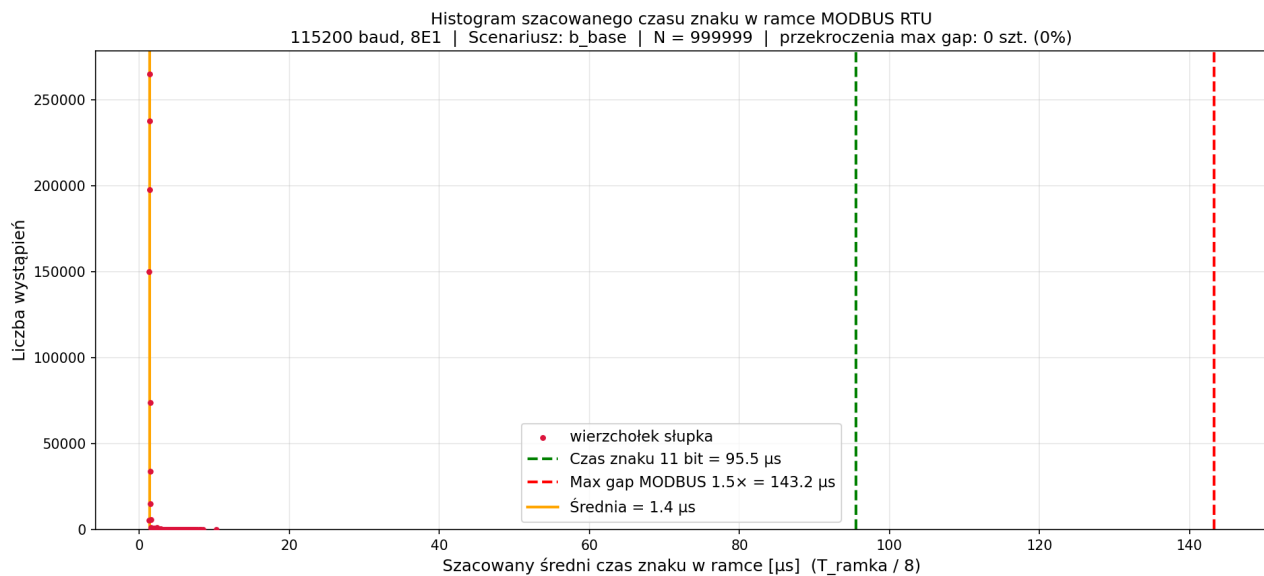
$$\begin{aligned}T_{znak} &\approx 95,49 \mu\text{s} && (\text{minimum: } 11 \text{ bitów} \times T_b) \\T_{ramka} &= 8 \cdot T_{znak} \approx 763,89 \mu\text{s} && (\text{minimum teoretyczne}) \\T_{gap_max} &= 1,5 \cdot T_{znak} \approx 143,23 \mu\text{s} && (\text{limit MODBUS RTU})\end{aligned}$$

a) Pomiar przy zamkniętych aplikacjach

Treść zadania

Dokonać pomiaru rozkładu czasów odstępu pomiędzy kolejnymi znakami w ramce metodą zastępczą. Sporządzić histogram częstości empirycznych. Przed uruchomieniem zamknąć wszystkie aktywne aplikacje.

Rozwiązanie



Rysunek 7: Histogram szacowanego czasu znaku w ramce MODBUS RTU — aplikacje zamknięte. Średni czas: $\bar{T}_{znak} = 1,4 \mu s$, przekroczenia limitu MODBUS: 0.

Szacowany średni czas znaku wynosi $1,4 \mu s$, co jest znacznie poniżej wartości teoretycznej $T_{znak} \approx 95,49 \mu s$. Wynika to z faktu, że pomiar odbywa się na wirtualnym porcie PTY, gdzie nie występuje rzeczywiste opóźnienie sprzętowego UART — czas mierzony to wyłącznie narzut programowy OS na wysłanie 8 bajtów. Nie odnotowano żadnych przekroczeń limitu MODBUS RTU, co oznacza, że w stanie spoczynku systemu ramka jest wysyłana płynnie i bez przerw wyłączeniowych.

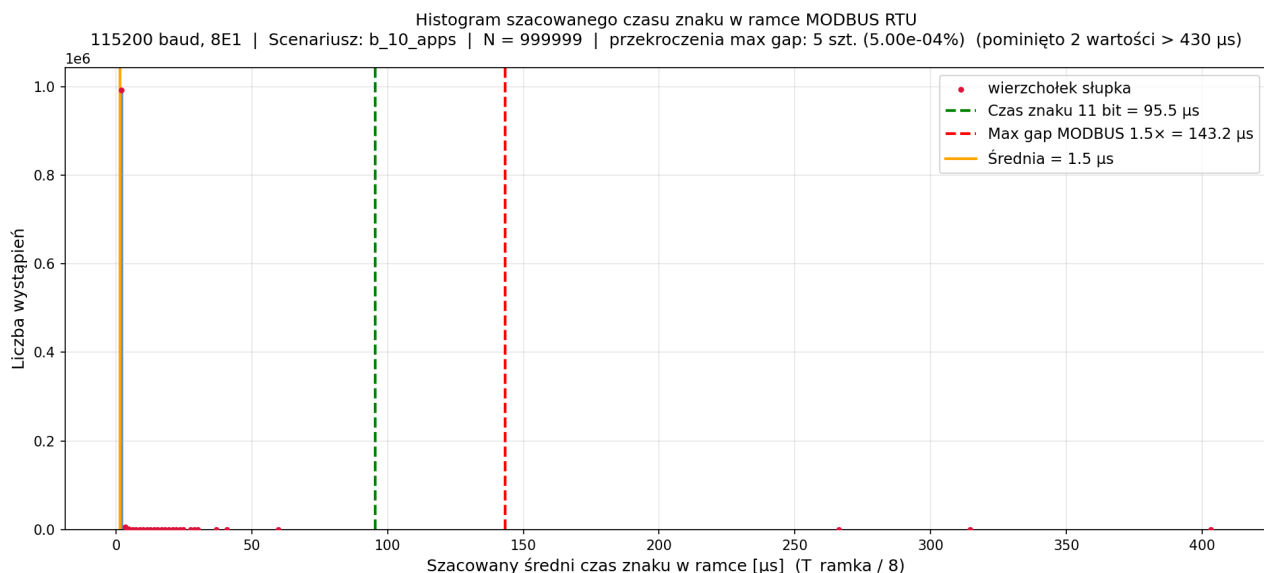
b) Pomiar przy 10 aktywnych aplikacjach

Treść zadania

Wyznaczyć rozkład czasów odstępów przy aktywnych 10 aplikacjach. Czy średni czas transmisji 1 znaku ulegnie zmianie?

Rozwiązanie

Uruchomiono 10 aplikacji na komputerze i powtórzono pomiar.



Rysunek 8: Histogram szacowanego czasu znaku w ramce MODBUS RTU — 10 aktywnych aplikacji. Średni czas: $\bar{T}_{znak} = 1,5 \mu$ s, przekroczenia limitu MODBUS: 5.

Aktywne aplikacje spowodowały wzrost średniego czasu o $\approx 7,1\%$ ($1,4 \rightarrow 1,5 \mu$ s). Pojawiło się 5 przekroczeń limitu MODBUS RTU (wcześniej: 0) — scheduler obciążony 10 procesami wyłącza sporadycznie proces pomiarowy podczas wysyłania ramki, co wprowadza przerwy rzędu setek mikrosekund.

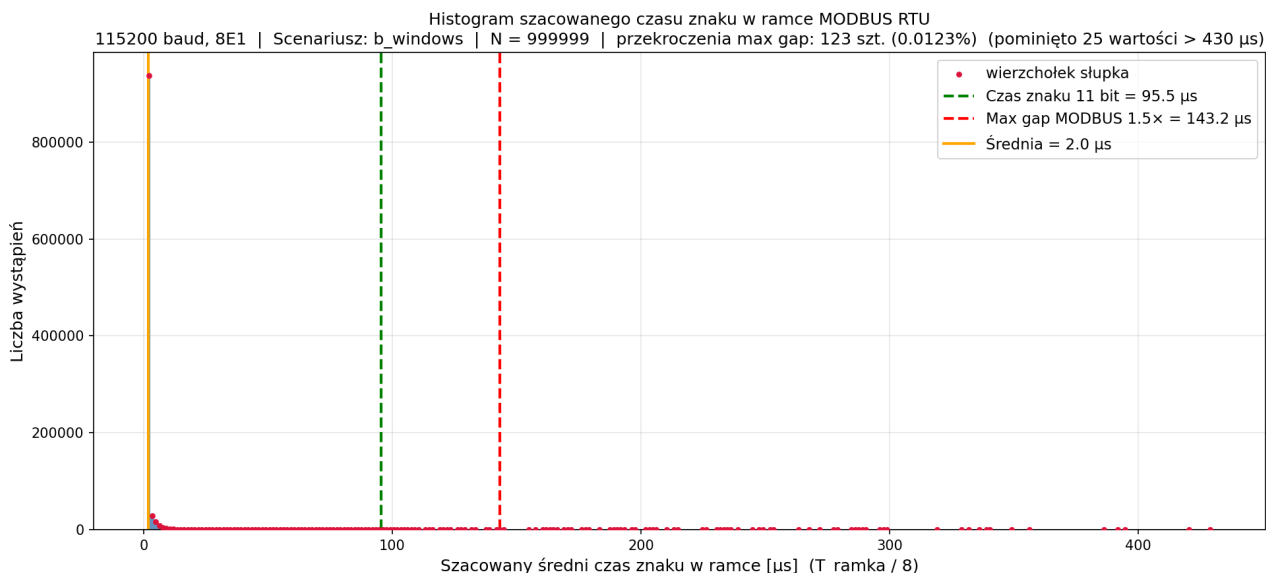
c) Wpływ operacji na oknach

Treść zadania

Zbadać wpływ ręcznego przeciągania i skalowania okien na średni czas odstępu między znakami w ramce.

Rozwiązanie

Podczas pomiaru wykonywano intensywne operacje na oknach systemu.



Rysunek 9: Histogram szacowanego czasu znaku w ramce MODBUS RTU — operacje na oknach. Średni czas: $\bar{T}_{znak} = 2,0 \mu s$, przekroczenia limitu MODBUS: 123.

Operacje na oknach spowodowały gwałtowne pogorszenie parametrów:

- Średni czas wzrósł o $\approx 42,9\%$ względem wartości bazowej ($1,4 \rightarrow 2,0 \mu s$).
- Liczba przekroczeń limitu MODBUS RTU wzrosła do **123** (z 0 w stanie bazowym i 5 przy 10 aplikacjach).

Przeciąganie okien angażuje compositor okienkowy i sterownik GPU, co powoduje długotrwałe wyłączenia procesu pomiarowego. Dla protokołu MODBUS RTU jest to krytyczne — każde z 123 przekroczeń odpowiada przerwie $> 143,23 \mu s$ w środku ramki, co odbiorca zinterpretowałby jako koniec transmisji i odebrał ramkę jako niekompletną lub błędną.

W zadaniu 2A metoda zastępcza (blok 10 bajtów) dała 5 przekroczeń podczas operacji na oknach, podczas gdy 2B (ramka 8 bajtów) dała 123. Bezpośrednie porównanie tych liczb jest jednak mylące — w obu przypadkach poruszano innymi aplikacjami: w 2A przesuwano okno edytora VSCode, natomiast w 2B okno komunikatora Discord. Discord jest aplikacją znacznie bardziej obciążającą GPU i compositor (renderowanie interfejsu webowego na Electron), co powoduje dłuższe i częstsze wyłączenia procesu pomiarowego. Wyniki potwierdzają tym samym, że rodzaj aplikacji działającej w tle ma istotny wpływ na determinizm czasowy transmisji.

System macOS, jako GPOS, nie gwarantuje terminowego wysyłania ramek MODBUS RTU przy obciążeniu systemu. Operacje graficzne powodują przekroczenia limitu inter-character gap, co w rzeczywistej instalacji przemysłowej oznaczałoby utratę spójności komunikacji. Aplikacje krytyczne czasowo wymagają RTOS lub dedykowanego sprzętowego kontrolera komunikacyjnego.

Implementacja

Do realizacji pomiarów napisano dwa skrypty w języku Python, współdzielące wspólną bibliotekę `shared.py` zawierającą stałe transmisji (T_b , T_{znak} , T_{gap_max}) oraz funkcje statystyczne i wizualizacyjne.

zadanie2a.py realizuje transmisję ciągłą bajtu `0xC4`. Dostępne są dwie metody pomiaru: metoda indywidualna, w której każdy bajt wysyłany jest osobno przez wywołanie `write(1B)` +

`flush()`, a czas między kolejnymi wywołaniami jest mierzony jako odstęp; oraz metoda blokowa (zastępcza), w której blok n bajtów wysyłany jest jednorazowo, a zmierzony czas całkowity dzielony jest przez n w celu uzyskania średniego czasu na bajt. Wyniki wizualizowane są jako histogram z liniami referencyjnymi T_{znak} i T_{gap_max} . Skrypt przyjmuje parametry przez CLI: port, liczba próbek, etykieta scenariusza, wybór metody oraz katalog wyjściowy.

zadanie2b.py realizuje transmisję ciągłą ramki MODBUS RTU 01 03 00 00 00 02 C4 0B (8 bajtów). Stosowana jest wyłącznie metoda zastępcza: mierzony jest czas całej ramki, który następnie dzielony jest przez liczbę znaków ($N = 8$), co daje szacowany średni czas na znak $\bar{T}_{znak} = T_{ramka}/8$. Wizualizacja i CLI analogiczne do **zadanie2a.py**; brak opcji metody blokowej.

Uruchamianie pomiarów odbywa się za pomocą skryptów powłoki `run_pomiar_2a.sh` i `run_pomiar_2b.sh`, które automatycznie tworzą wirtualną parę portów szeregowych przez `socat`, wykrywają ścieżki PTY i uruchamiają drainer bufora na drugim porcie. Wspólna logika (uruchomienie `socat`, oczekiwanie na inicjalizację portów, obsługa `cleanup`) wydzielona jest do `shared.sh`.

Dodatek A — Wywłaszczalność zadań

Pojęcie wywłaszczalności

Wywłaszczalność (*preemptibility*) oznacza zdolność systemu operacyjnego do przerywania aktualnie wykonywanego zadania w dowolnym momencie i przekazania procesora innemu zadaniu o wyższym priorytecie. W systemach z wywłaszczaniem wielozadaniowym (*preemptive multitasking*) — do których należą macOS i Windows — scheduler może wywłaszczyć dowolny proces w każdej chwili, niezależnie od tego, czy jest on gotowy na takie przerywanie.

Wpływ na pomiary

Każdy skok na histogramie powyżej wartości typowej jest bezpośrednim śladem wywłaszczenia procesu pomiarowego. Sekwencja zdarzeń:

1. Proces pomiarowy wywołuje `write()` i `flush()`, po czym scheduler uznaje, że inne zadanie OS ma wyższy priorytet.
2. Proces zostaje zawieszony na czas kwantu schedulera lub dłużej, jeśli inne zadania utrzymują wysoki priorytet.
3. Po powrocie do wykonania proces mierzy czas — różnica zawiera całe opóźnienie wywłaszczenia.

Stąd wynikają obserwacje z pomiarów:

- **Ogon prawostronny histogramu** — rzadkie, duże wartości odpowiadają długim wywłaszczeniom (np. przez podsystem graficzny).
- **Wzrost liczby przekroczeń limitu MODBUS z 2 do 86** przy operacjach na oknach — compositor i sterownik GPU przejmują procesor na czas renderowania klatki.
- **Metoda blokowa nie eliminuje wywłaszczenia** — jeśli wywłaszczenie nastąpi podczas wywołania `write(block)`, jego koszt jest jedynie $1/n$ -krotnie mniejszy, nie zerowy.

GPOS vs. RTOS

Cecha	GPOS (macOS / Windows)	RTOS
Polityka schedulera	priorytet + kwant czasu	priorytet ściśle hierarchiczny
Wyłączenie zadania RT	możliwe przez zadania OS	tylko przez wyższy priorytet
Maksymalne opóźnienie	nieokreślone	gwarantowane (WCET)
Źródło jitteru	ISR, compositor, GC, sieć	tylko przerwania wyższego priorytetu

W systemie RTOS zadanie RT przypisane do odpowiedniego priorytetu nie zostałoby wyłączone przez podsystem graficzny, co wyeliminowałoby prawy ogon histogramu i przekroczenia limitu MODBUS.

Dodatek B — Podstawy teoretyczne: Zadanie 2B

Ramka MODBUS RTU

Ramka (ang. *frame*) to niepodzielna sekwencja znaków traktowana przez protokół jako jedna jednostka komunikacji. W protokole MODBUS RTU granice ramki wyznaczone są przez ciszę na łączu dłuższą niż $3,5 \times T_{znak}$ (przed ramką) i $1,5 \times T_{znak}$ między kolejnymi znakami wewnątrz ramki.

Badana ramka to zapytanie jednostki nadrzędnej o odczyt 2 rejestrów Holding:

Bajt(y)	Wartość (hex)	Znaczenie
1	01	Adres urządzenia slave (1)
2	03	Kod funkcji: Read Holding Registers
3–4	00 00	Adres startowy rejestru (0)
5–6	00 02	Liczba rejestrów do odczytu (2)
7–8	C4 0B	Suma kontrolna CRC-16 (little-endian: 0x0BC4)

Znak i ramka znakowa (11 bitów)

Każdy bajt transmitowany jest jako *znak* złożony z 11 bitów na łączu (format 8E1):

$$1 \text{ (start)} + 8 \text{ (dane)} + 1 \text{ (parzystość)} + 1 \text{ (stop)} = 11 \text{ bitów}$$

$$T_b = \frac{1}{115200} \approx 8,68 \mu s$$

$$T_{znak} = 11 \cdot T_b \approx 95,49 \mu s$$

$$T_{ramka} = 8 \cdot T_{znak} \approx 763,89 \mu s \quad (\text{minimum teoretyczne})$$

Inter-character gap to cisza między końcem jednego znaku a początkiem następnego. Limit MODBUS RTU: $\text{gap} > 1,5 \times T_{znak} \approx 143,23 \mu s$ jest interpretowany jako koniec ramki — odbiorca uznaje transmisję za przerwana.

Metoda zastępcza pomiaru

Bezpośredni pomiar każdego odstępu wymagałby sprzętowego timestampowania na poziomie bitu (oscyloskop lub analizator protokołów). W ramach metody zastępczej mierzy się czas całej ramki T_{ramka} i szacuje się średni czas na znak:

$$\bar{T}_{znak} = \frac{T_{ramka}}{N_{znakw}} = \frac{T_{ramka}}{8}$$